Reviewer Pack — Minimal Index of Quantum Entanglement over Noncommutative Ge…

Entry 7b2eab27753e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 00:21:56 UTC

Minimal Index of Quantum Entanglement over Noncommutative Geometry

Entry ID: `7b2eab27753e` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-26 00:21:56 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory (Entanglement) **Field B** (complexity object): Noncommutative Geometry

Statement:

> ['For any non-commutative algebra A, the minimal index of entanglement for an n-qubit state is upper bounded by the degree of the first non-zero eigenvalue in the spectrum of A.', 'The conjecture states that for a given quantum system with n qubits, the degree of the algebraic object representing its non-commutative geometry (as a spectral problem) is at most equal to the minimal index of entanglement among all possible pure states of that system.', 'This implies that the complexity of determining the entanglement in a system is bounded by the algebraic structure of the corresponding quantum state.']

Rationale (proposer's reasoning):

> ['Quantum information theory has long been known to be related to the non-commutative geometry due to their shared mathematical structures and properties. This conjecture bridges these two fields by proposing a direct relationship between an entanglement measure and an algebraic invariant, which has not been explored before.', 'If true, this would provide new insights into both quantum information theory and noncommutative geometry, potentially leading to new algorithms for studying entangled systems or proving lower bounds on their complexity.']

Taxonomy category: `SOS_DEGREE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `4198dbd92c21145e`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, across 30 random seeds, the mean of the ratio of the minimal index of entanglement to the degree of the first non-zero eigenvalue in the algebraic spectrum is less than or equal to 1.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "noncommutative geometry" AND "quantum entanglement" AND minimal index" - "algebraic structure" AND "entanglement complexity" AND noncommutative algebra" - degree of eigenvalue spectrum AND bounds ON entanglement index

Top relevant hits considered: - [http://arxiv.org/abs/1410.8670v1] Free-fermion Entanglement Spectrum through Wannier Interpolation - [http://arxiv.org/abs/1405.7816v3] Fluctuations and Entanglement spectrum in quantum Hall states - [http://arxiv.org/abs/1108.2907v1] Trace Index and Spectral Flow in the Entanglement Spectrum

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def matrix_multiplication(A, B):
        n = len(A)
        C = [[0 for _ in range(n)] for _ in range(n)]
        for i in range(n):
            for j in range(n):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def identity_matrix(n):
        I = [[0 for _ in range(n)] for _ in range(n)]
        for i in range(n):
            I[i][i] = 1
        return I

    def gaussian_elimination(A, b):
        n = len(A)
        for i in range(n):
            max_row = i + max(range(i, n), key=lambda k: abs(A[k][i]))
```

```
            A[i], A[max_row] = A[max_row], A[i]
            b[i], b[max_row] = b[max_row], b[i]
            for j in range(i + 1, n):
                factor = A[j][i] / A[i][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]
                b[j] -= factor * b[i]
        x = [0 for _ in range(n)]
        for i in range(n - 1, -1, -1):
            x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i + 1, n))) / A[i][i]
        return x

    def minimal_index_of_entanglement(state):
        n = len(state)
        identity = identity_matrix(n)
        A = matrix_multiplication(state, state) - identity
        eigenvalues = []
        for i in range(100):  # Perform a few iterations of Gaussian elimination to approximate ei
            b = [random.random() for _ in range(n)]
            x = gaussian_elimination(A, b)
            lambda_i = sum(x[j] * state[j][i] for j in range(n))
            eigenvalues.append(abs(lambda_i))
        return min(eigenvalues) if eigenvalues else float('inf')

    def degree_of_first_nonzero_eigenvalue(state):
        n = len(state)
        identity = identity_matrix(n)
        A = matrix_multiplication(state, state) - identity
        eigenvalues = []
        for i in range(100):  # Perform a few iterations of Gaussian elimination to approximate ei
            b = [random.random() for _ in range(n)]
            x = gaussian_elimination(A, b)
            lambda_i = sum(x[j] * state[j][i] for j in range(n))
            eigenvalues.append(abs(lambda_i))
        nonzero_eigenvalues = [lambda_ for lambda_ in eigenvalues if lambda_ > 1e-6]
        return min(nonzero_eigenvalues) if nonzero_eigenvalues else float('inf')

    state = [[random.random() for _ in range(4)] for _ in range(4)]
    min_index = minimal_index_of_entanglement(state)
    degree = degree_of_first_nonzero_eigenvalue(state)

    return {
        "metric_name": "Ratio of Minimal Index to Eigenvalue Degree",
        "metric_value": min_index / (degree + 1e-6),
        "instances_tested": 1,
        "conjecture_holds": min_index <= degree,
        "counterexample": "" if min_index <= degree else "Mapping undefined"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
```

```
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='Mapping undefined' first_failing_seed={first_fai
    else:
        print("RESULT: INCONCLUSIVE Reason=All trials used n=1")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8028d7ef.py", line 95, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8028d7ef.py", line 78, in run_trial
    min_index = minimal_index_of_entanglement(state)
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8028d7ef.py", line 55, in minimal_index
    A = matrix_multiplication(state, state) - identity
        ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^~~~~~~~~~
TypeError: unsupported operand type(s) for -: 'list' and 'list'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which prevents us from verifying the conjecture's support condition. | next: Debug the test code to ensure it runs without errors and produces the required data for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11934
2	preregistration	ollama_remote	glm4:latest	0	0	5200
3	novelty	ollama_remote	glm4:latest	0	0	4738
4	novelty	ollama_remote	glm4:latest	0	0	5740
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15786
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11258
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12165
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12832
9	judge	ollama_remote	glm4:latest	0	0	8941

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 88595 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/7b2eab27753e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7b2eab27753e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/7b2eab27753e.tar.gz` (if generated)